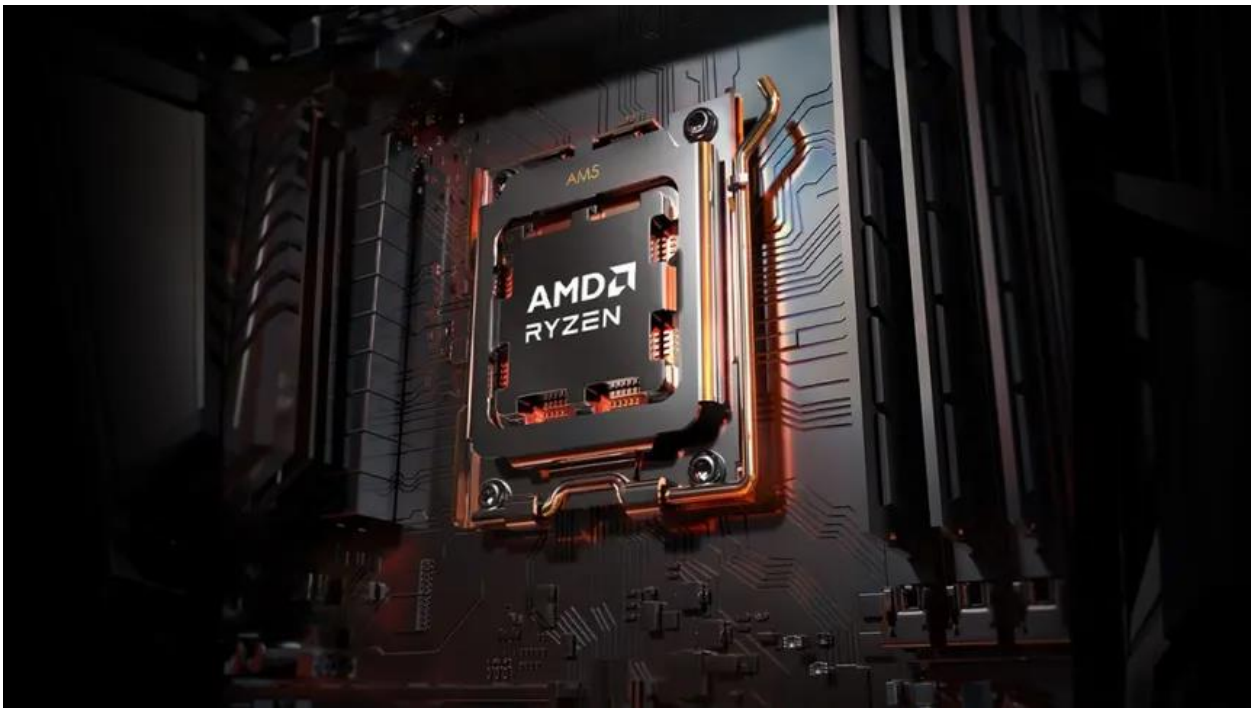
Análisis de Algoritmos
2026/2027

Práctica 1

Santiago Pita y Matias Cantalejo, Grupo 12.



1. Introducción

Esta primera práctica de la asignatura de Análisis de Algoritmos tiene como objetivo asentar las bases metodológicas y experimentales para el estudio del coste computacional. El trabajo se centra en el diseño, implementación y evaluación empírica de dos algoritmos de ordenación: el método por inserción (InsertSort) y el método de la burbuja (BubbleSort). Para garantizar la fiabilidad del análisis, se introduce el estudio del algoritmo de mezcla de Fisher-Yates para la generación de permutaciones aleatorias equiprobables, asegurando que las muestras de prueba no introduzcan sesgos estadísticos que puedan invalidar los resultados del laboratorio.

La discusión previa de la práctica gira en torno a la validación del modelo teórico mediante la experimentación real. Mientras que el análisis matemático proporciona una aproximación asintótica del comportamiento de los algoritmos en el caso medio, la medición de los tiempos reales de procesamiento de la CPU y el recuento exhaustivo de las operaciones básicas en un entorno Unix permiten comprobar el impacto de las constantes ocultas y de los detalles de bajo nivel, como la gestión de la memoria dinámica y la eficiencia de los desplazamientos frente a los intercambios físicos de elementos. De este modo, la práctica conecta la teoría algorítmica con la ingeniería del software y los estándares de programación.

A nivel técnico, los programas reciben como entrada los tamaños de los vectores a evaluar y la cantidad de permutaciones a promediar, y generan como salida ficheros de registro con los tiempos medios de CPU y la cuenta (medio, máximo y mínimo) de las operaciones básicas ejecutadas.

2. Herramientas y metodología

Para el desarrollo de esta práctica se ha utilizado el Ubuntu, empleado tanto en los ordenadores de los laboratorios de la universidad como en los equipos personales en casa. Como entorno de edición del código fuente en lenguaje C se ha utilizado Visual Studio Code. La compilación y el enlazado de los módulos se han gestionado mediante la herramienta Make, utilizando el compilador oficial de proyectos GNU (gcc) y el estándar C99. Para el análisis visual y la representación gráfica de las curvas de tiempos y operaciones se ha empleado la herramienta Gnuplot. La redacción, maquetación y discusión teórica del documento de la memoria se han llevado a cabo de forma colaborativa combinando Microsoft Word.

La metodología de desarrollo ha seguido un enfoque modular y progresivo. En primer lugar, se crearon las funciones básicas de soporte para el control de la aleatoriedad. Posteriormente, se implementó el motor de ordenación, y finalmente se estructuró la lógica de captura de tiempos del procesador. El criterio fundamental en la solución del problema ha sido la modularidad y el aislamiento de recursos, garantizando que cada función gestione y libere sus propias reservas de memoria en caso de error para evitar fugas.

Las pruebas realizadas han abarcado tanto la verificación de la lógica algorítmica como la robustez frente a fallos. Se han ejecutado pruebas unitarias con vectores de tamaños reducidos para certificar el orden correcto de los elementos en pantalla, comprobaciones con segmentos intermedios de los arrays que no comenzaban en el índice cero, y casos extremos como llamadas con argumentos no

válidos. Finalmente, se realizaron pruebas de carga masiva automatizadas desde el gestor de compilación para validar que los tiempos y el contador de operaciones básicas se comportan de manera estable y predecible al escalar el tamaño del problema.

También se ha utilizado https://hilite.me/ para darle color al código y también Gemini para formatear el código ya que al pasar de Visual Studio a Word por cambios en el formato se añaden espacios extra.

Las gráficas se han generado en una máquina virtual de Ubuntu en un ordenador que corre en Windows 10 con un Ryzen 5 3600 y 8GB de RAM (solo para la máquina virtual). Los tamaños analizados van de 100 a 2000 y el número de permutaciones promedio es de 500, y la semilla utilizada ha sido srand(time(NULL)) por lo que cada vez que se corre el programa la semilla es distinta y produce un resultado distinto.

3. Apartados

Apartado 1

```c
int random_num(int inf, int sup) {
    if (sup < inf || RAND_MAX < (sup - inf)) {
        return ERR;
    }

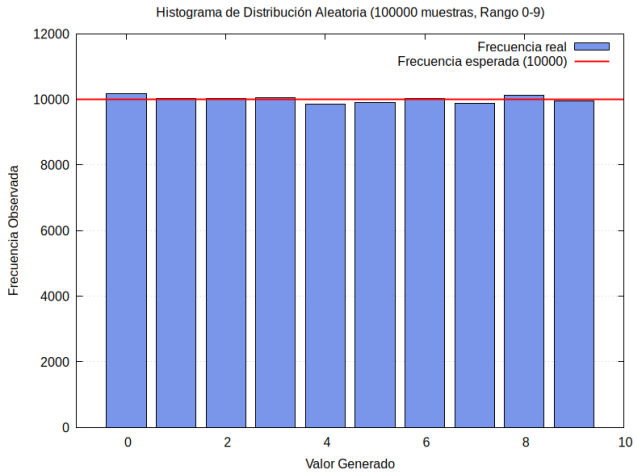
    return inf + rand() % (sup - inf + 1);
}
```

Explicación del desarrollo

- Validación de errores: Se comprueba inicialmente que el límite inferior sea menor o igual al superior (sup < inf). Además, se verifica que la amplitud del intervalo no supere la capacidad de RAND_MAX para evitar comportamientos indefinidos o desbordamientos enteros, devolviendo la macro ERR si los argumentos no son válidos.
- Generación del número: Se utiliza la función estándar rand() de la biblioteca de C. El valor obtenido se proyecta al tamaño del intervalo deseado utilizando el operador módulo (sup - inf + 1) y se desplaza sumando el límite inferior (inf), garantizando que el resultado se mantenga estrictamente dentro de las cotas solicitadas.

Resultados y análisis de distribución

- Parámetros de la prueba: Para la validación empírica de la función, se ha generado una muestra de 100000 números aleatorios en el intervalo cerrado $[0,9]$.
- Análisis visual: Como se observa en el histograma adjunto, la frecuencia observada para cada entero del intervalo se mantiene cercana a la frecuencia teórica esperada. Aunque se aprecian ligeras oscilaciones propias de la varianza estadística de una muestra finita, la distribución tiende a la uniformidad.



Apartado 2

```c
int* generate_perm(int N) {
    int *array = NULL;
    int i = 0;
    int temp = 0;
    int random = 0;

    if (N <= 0) {
        return NULL;
    }

    array = (int*)calloc(N, sizeof(int));
    if (!array) {
        /* error */
        return NULL;
    }

    for (i = 0; i < N; i++) {
        array[i] = i + 1;
    }

    for (i = 0; i < N; i++) {
        random = random_num(i + 1, N);
        if (random == ERR) {
            free(array);
            return NULL;
        }
        /* intercambiar perm[i] con perm[random_num(i, N)] */
        temp = array[i];
        array[i] = array[random - 1];
        array[random - 1] = temp;
    }

    return array;
}
```

Explicación del desarrollo

- Gestión de memoria y errores: Se valida que la longitud sea positiva (N > 0) y se reserva el array con calloc. Si la reserva inicial o la generación aleatoria posterior fallan (random == ERR), se devuelve NULL asegurando la liberación de la memoria con free(array) para evitar memory leaks.
- Inicialización: Dado que los índices en C van de 0 a N-1, el array se inicializa asignando array[i] = i + 1 para almacenar correctamente los valores del 1 al N.
- Algoritmo de Fisher-Yates (Adaptación de índices): Para la fase de mezcla, la llamada random_num(i + 1, N) genera el número en el rango lógico. Para realizar el intercambio,

este valor se ajusta al índice real del array de C restándole 1 (random - 1) y se hace el swap usando la variable temporal temp.

Resultados de la ejecución

- Verificación con el entorno de pruebas: Para asegurar la validez de la implementación, se ha compilado y ejecutado el módulo de comprobación exercise2.c tal y como se solicita en el guion de la práctica.
- Análisis de la permutación: Al ejecutar el programa indicando un tamaño concreto por consola (por ejemplo, N = 10), la salida estándar imprime el array generado. Se ha comprobado que la tabla resultante contiene todos los números naturales del 1 al N desordenados, sin repeticiones ni omisiones.
- Conclusión: Este resultado empírico garantiza que la traducción del pseudocódigo a C es correcta y que la delicada adaptación de los índices (para que iteren de 0 a N-1 en C mientras los valores lógicos van de 1 a N) no ha introducido desbordamientos de buffer ni fallos de segmentación.

Apartado 3

```c
int** generate_permutations(int n_perms, int N) {
    int i = 0;
    int j = 0;
    int **matrix;
    matrix = NULL;

    if (N <= 0 || n_perms <= 0) {
        return NULL;
    }

    matrix = (int**)calloc(n_perms, sizeof(int*));
    if (!matrix) {
        return NULL;
    }

    for (i = 0; i < n_perms; i++) {
        /*
        matrix[i] = (int*)calloc(1, sizeof(int));
        if (!matrix[i]) {
            for (j = i; j >= 0; j--) {
                free(matrix[j]);
            }
            free(matrix);
            return NULL;
        }
        */

        matrix[i] = generate_perm(N);
        if (!matrix[i]) {
            for (j = 0; j < i; j++) {
                free(matrix[j]);
            }
            free(matrix);
            return NULL;
        }
    }

    return matrix;
}
```

Explicación del desarrollo

- Gestión de la matriz dinámica: Se valida que ambos argumentos sean estrictamente positivos. A continuación, se reserva memoria mediante calloc para un array principal de punteros (int**) con capacidad para alojar n_perms permutaciones.
- Generación independiente: Se utiliza un bucle que invoca secuencialmente a la función generate_perm(N). Al almacenar directamente el retorno de esta función en matrix[i], cada permutación se genera de forma completamente independiente de las demás, admitiendo repeticiones lógicas dentro del conjunto final.
- Manejo robusto de errores (Memory Leaks): El código implementa una estrategia de retroceso (rollback) muy estricta. Si la reserva de una permutación falla en la iteración *i*, el algoritmo no aborta de inmediato, sino que recorre hacia atrás el array liberando explícitamente las permutaciones de *0* a *i-1*, para finalmente liberar el array principal y devolver NULL. Esto garantiza un control hermético de los recursos en caso de fallo crítico.

Resultados de la ejecución

- Verificación con el entorno de pruebas: El correcto funcionamiento de esta función se ha certificado compilando y ejecutando el programa suministrado exercise3.c.
- Análisis del volcado de memoria: Durante las pruebas, se comprobó que el programa principal es capaz de recorrer el doble puntero retornado y mostrar por pantalla múltiples permutaciones del tamaño solicitado. Asimismo, el programa de prueba finaliza limpiamente tras liberar cada vector individual y la matriz principal, demostrando que la asignación de memoria realizada en el interior de la función respeta los estándares y evita la creación de bloques de memoria inaccesibles (segmentation fault).

Apartado 4

```
int InsertSort(int* array, int ip, int iu) {
    int i, j, key;
    int count_ob = 0;
    int seguir;

    if (!array || ip < 0 || ip > iu) {
        return ERR;
    }

    for (i = ip + 1; i <= iu; i++) {
        key = array[i];
        j = i - 1;
        seguir = 1;

        while (j >= ip && seguir) {
            count_ob++;

            if (array[j] > key) {
                array[j + 1] = array[j];
                j--;
            } else {
                seguir = 0;
            }
        }
        array[j + 1] = key;
    }

    return count_ob;
}
```

Explicación del algoritmo y de la Operación Básica (OB)

Operación Básica: La Operación Básica (OB) seleccionada para este algoritmo es la comparación de elementos del array, concretamente la instrucción array[j] > key. Se ha elegido esta operación porque es la que determina si se realiza o no un desplazamiento de elementos y domina el tiempo de ejecución en el bucle interior.

Conteo de la OB: Para llevar la cuenta de todas las evaluaciones, incluidas aquellas que produzcan un resultado falso (es decir, cuando array[j] <= key), el incremento del contador (count_ob++) se ha colocado inmediatamente después de entrar en el bucle while.

La lógica implementada funciona de la siguiente manera:

1. El bucle for principal recorre la tabla desde el segundo elemento del segmento (ip + 1) hasta el último (iu).
2. Se guarda el elemento actual en la variable key.
3. El bucle while retrocede por los elementos anteriores (índice j). Justo al iniciar la iteración del while, se asume que se va a evaluar la comparación array[j] > key, por lo que se incrementa count_ob en 1.
4. A continuación, se evalúa la condición:
    a. Si array[j] > key es cierto, se desplaza el elemento a la derecha (array[j + 1] = array[j]) y se decremente j.
    b. Si es falso (incluyendo el caso de que el elemento ya esté en su posición correcta), se entra en el else y se pone la bandera seguir = 0. Esto fuerza la salida del bucle while en la siguiente comprobación de la condición de parada, garantizando que el contador registró esa evaluación fallida antes de interrumpir la búsqueda.
5. Se inserta el key en su posición definitiva (array[j + 1] = key).

Resultados de la ejecución y pruebas de robustez

Tal y como se estipula en el guion de la práctica, se ha adaptado el fichero de pruebas exercise4.c sustituyendo la llamada original por nuestra función InsertSort. Las pruebas realizadas garantizan el correcto funcionamiento del algoritmo bajo distintas condiciones:

- **Segmentos estándar:** Se generaron tablas aleatorias con índices desde ip = 0 hasta iu = N-1. Al imprimir el resultado por consola, se verificó visualmente que todos los elementos quedaban ordenados de menor a mayor.
- **Segmentos intermedios:** Se ejecutó la ordenación sobre rangos internos de la tabla (por ejemplo, con ip = 2 e iu = 5). Se comprobó que el algoritmo reordenaba exclusivamente los elementos de ese segmento, dejando los elementos anteriores (i < ip) y posteriores (i > iu) completamente intactos en sus posiciones originales.
- **Casos extremos y control de errores:**
    - **Tablas de un solo elemento:** Se forzó la llamada con ip = iu. El bucle principal (i = ip + 1) no se ejecuta al no cumplirse la condición i <= iu, retornando 0 ejecuciones de la Operación Básica y dejando el elemento intacto, lo cual es el comportamiento matemáticamente correcto ya que una tabla de un elemento está trivialmente ordenada.
    - **Argumentos no válidos:** Se simuló el paso de un puntero NULL y de índices incoherentes (como ip < 0 o ip > iu). En todos estos escenarios, el sistema de control de errores interceptó el fallo en la cabecera de la función, abortando la ejecución y retornando limpiamente el código de error ERR.

Apartado 5

```
short average_sorting_time(pfunc_sort metodo,
                           int n_perms,
                           int N,
                           PTIME_AA ptime) {

    if (!metodo || n_perms <= 0 || N <= 0 || !ptime) {
        return ERR;
    }

    int i = 0;
    int j = 0;
    clock_t inicio, fin;
    int **array = NULL;
    int obs = 0;
    int max_obs = 0;
    int min_obs = 0;
    long total_obs = 0;
    double total_time = 0.0;

    array = generate_permutations(n_perms, N);
    if (!array) {
        return ERR;
    }

    for (i = 0; i < n_perms; i++) {

        inicio = clock();
        obs = metodo(array[i], 0, N - 1);
        fin = clock();

        if (obs == ERR) {
            for (j = 0; j < n_perms; j++) {
                free(array[j]);
            }
            free(array);
            return ERR;
        }

        total_time += (double)(fin - inicio) / CLOCKS_PER_SEC; /* casting para
que devuelva todo */

        if (i == 0) {
            min_obs = obs;
            max_obs = obs;
        } else {
            if (obs < min_obs) {
                min_obs = obs;
            }
```

```
            if (obs > max_obs) {
                max_obs = obs;
            }
        }
        total_obs += obs;
    }

    ptime->N = N;
    ptime->n_elems = n_perms;
    ptime->time = total_time / n_perms;
    ptime->average_ob = (double)total_obs / n_perms;
    ptime->max_ob = max_obs;
    ptime->min_ob = min_obs;

    for (i = 0; i < n_perms; i++) {
        free(array[i]);
    }

    free(array);

    return OK;
}

short generate_sorting_times(pfunc_sort method, char* file,
                             int num_min, int num_max,
                             int incr, int n_perms) {
    if (!method || !file || num_min <= 0 || num_max <= 0 || incr <= 0 || n_perms
<= 0 || num_min > num_max) {
        return ERR;
    }

    int n_times = 0;
    int current_N = 0;
    int i = 0;
    PTIME_AA table = NULL;
    short status = OK;

    n_times = ((num_max - num_min) / incr) + 1;

    table = (PTIME_AA)calloc(n_times, sizeof(TIME_AA));
    if (!table) {
        return ERR;
    }

    current_N = num_min;

    for (i = 0; current_N <= num_max; i++) {
        status = average_sorting_time(method, n_perms, current_N, &table[i]);
        if (status == ERR) {
            free(table);
```

```
            return ERR;
        }
        current_N += incr;
    }

    status = save_time_table(file, table, n_times);

    free(table);

    return status;
}

short save_time_table(char* file, PTIME_AA ptime, int n_times) {
    if (!file || !ptime || n_times <= 0) {
        return ERR;
    }

    int i = 0;
    FILE *f = NULL;

    f = fopen(file, "w");
    if (!f) {
   return ERR;
    }

    /* una fila por tamaño, 6 columnas */
    fprintf(f, "# N     |  n_elems  |     time     |     average_ob     |
min_ob     |     max_ob\n");
    for (i = 0; i < n_times; i++) {
        fprintf(f, "  %-5d | %-9d | %-12.6f | %-18.2f | %-12d | %-12d\n",
                ptime[i].N, ptime[i].n_elems, ptime[i].time, ptime[i].average_ob,
ptime[i].min_ob, ptime[i].max_ob);
    }

    fclose(f);

    return OK;
```

Explicación del desarrollo

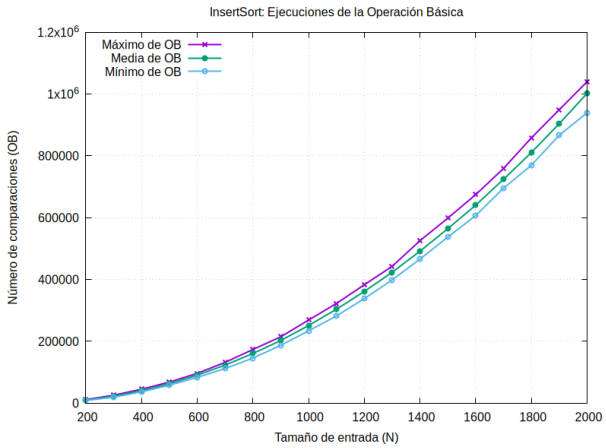
- Aislamiento del cronometraje: En la función average_sorting_time, la medición del tiempo mediante la función clock() envuelve estricta y exclusivamente a la llamada del

algoritmo de ordenación. La generación previa de las permutaciones aleatorias y la posterior liberación de memoria se han excluido deliberadamente del intervalo medido para no falsear el coste real de la CPU.

- Precisión aritmética: Para evitar truncamientos por división entera y pérdida de resolución, la diferencia de tiempos (fin - inicio) se proyecta a tipo double antes de dividirse por CLOCKS_PER_SEC. Asimismo, para el sumatorio de las Operaciones Básicas se ha empleado una variable de tipo long (total_obs), evitando así desbordamientos enteros al acumular miles de evaluaciones.
- Cálculo de extremos estadísticos: Los valores min_obs y max_obs se inicializan tomando como referencia absoluta la primera ordenación del lote (i == 0). En las iteraciones subsiguientes, se actualizan dinámicamente mediante comprobaciones condicionales si el recuento actual supera los umbrales registrados.
- Volcado seguro en disco: La función save_time_table verifica la correcta apertura del fichero en modo escritura ("w"). Posteriormente, formatea las estadísticas calculadas en las seis columnas exactas exigidas por la práctica (N, n_elems, time, average_ob, min_ob y max_ob) asegurando el cierre del descriptor del archivo al finalizar.

Resultados empíricos

- Verificación: Se compiló y ejecutó el programa exercise5_insertsort.c parametrizado adecuadamente. El programa generó correctamente el archivo de registro con las estadísticas de la ordenación por inserción.
- Como se puede apreciar en la siguiente gráfica y en el extracto de la tabla de datos, el tiempo medio de CPU y el volumen de operaciones básicas crecen de forma coherente al incrementar el tamaño de la entrada.



insert.log

# N	n_elems	time	average_ob	min_ob	max_ob
100	500	0.000012	2570.59	2058	3126
200	500	0.000032	10128.81	9044	11509
300	500	0.000060	22718.22	20033	25847
400	500	0.000104	40339.57	35798	44772
500	500	0.000158	62854.98	56887	68431
600	500	0.000230	90438.12	82787	97085
700	500	0.000329	123037.34	113883	131899
800	500	0.000381	160705.85	146931	174038
900	500	0.000555	202752.68	187497	215568
1000	500	0.000739	250486.69	235858	265302
1100	500	0.000805	303372.46	285283	321715
1200	500	0.000952	360209.58	339995	381404
1300	500	0.001127	423412.39	404164	447928
1400	500	0.001220	491195.24	466178	521487
1500	500	0.001570	563585.71	532489	592174
1600	500	0.001630	641529.84	613328	679940
1700	500	0.001766	723165.77	692415	754551
1800	500	0.001937	812505.43	778788	852495
1900	500	0.002182	906303.13	866074	944757
2000	500	0.002511	1002005.08	962363	1039675

Apartado 6

```
int BubbleSort(int* array, int ip, int iu) {
    int i, j, key;
    int count_ob = 0;
    int seguir;

    if (!array || ip < 0 || ip > iu) {
        return ERR;
    }

    i = iu;
    seguir = 1;

    while (i > ip && seguir) {
        seguir = 0;

        for (j = ip; j < i; j++) {
            count_ob++;

            if (array[j] > array[j + 1]) {
                key = array[j];
                array[j] = array[j + 1];
                array[j + 1] = key;

                seguir = 1;
            }
        }

        i--;
    }

    return count_ob;
}
```

Explicación del desarrollo y Metodología

- Variante implementada (Parada anticipada): Se ha implementado la variante de la burbuja optimizada. Se utiliza la variable seguir para detener el algoritmo si se detecta que el array ya está ordenado. Al inicio de cada pasada se asume que lo está (seguir = 0). Si se realiza algún intercambio en el bucle interior, la bandera cambia a 1. Si finaliza una pasada completa sin registrar intercambios, el bucle exterior se interrumpe de inmediato.
- Operación Básica (OB): La OB seleccionada es la comparación de elementos adyacentes (array[j] > array[j+1]). El contador count_ob++ se incrementa justo antes de la instrucción if para asegurar que se registren absolutamente todas las evaluaciones, independientemente de si resultan verdaderas (provocando un intercambio) o falsas.

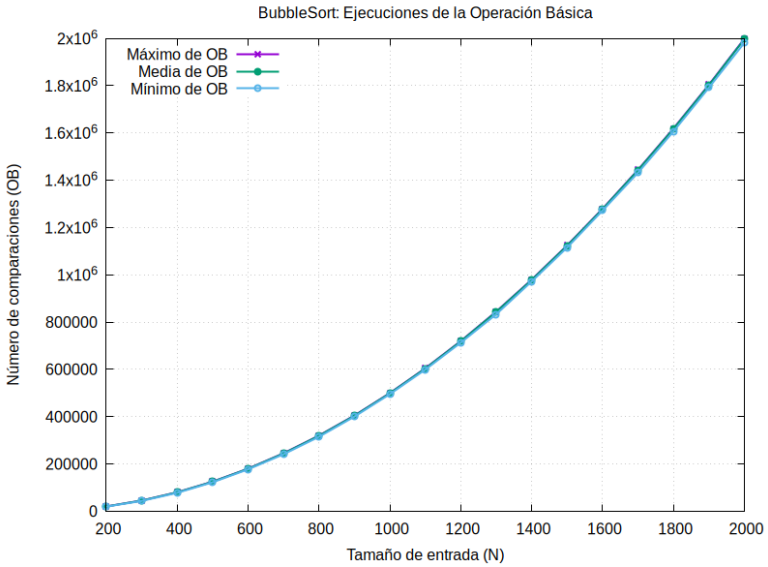
- Metodología de prueba: Se adaptaron los ficheros exercise4.c y exercise5.c para evaluar ambos algoritmos secuencialmente. Para garantizar un análisis empírico justo, las comparaciones se realizaron sobre copias idénticas de las mismas permutaciones sin ordenar, manteniendo constantes las condiciones del sistema y las opciones de compilación, y derivando las métricas a ficheros de registro independientes (insert.log y bubble.log).

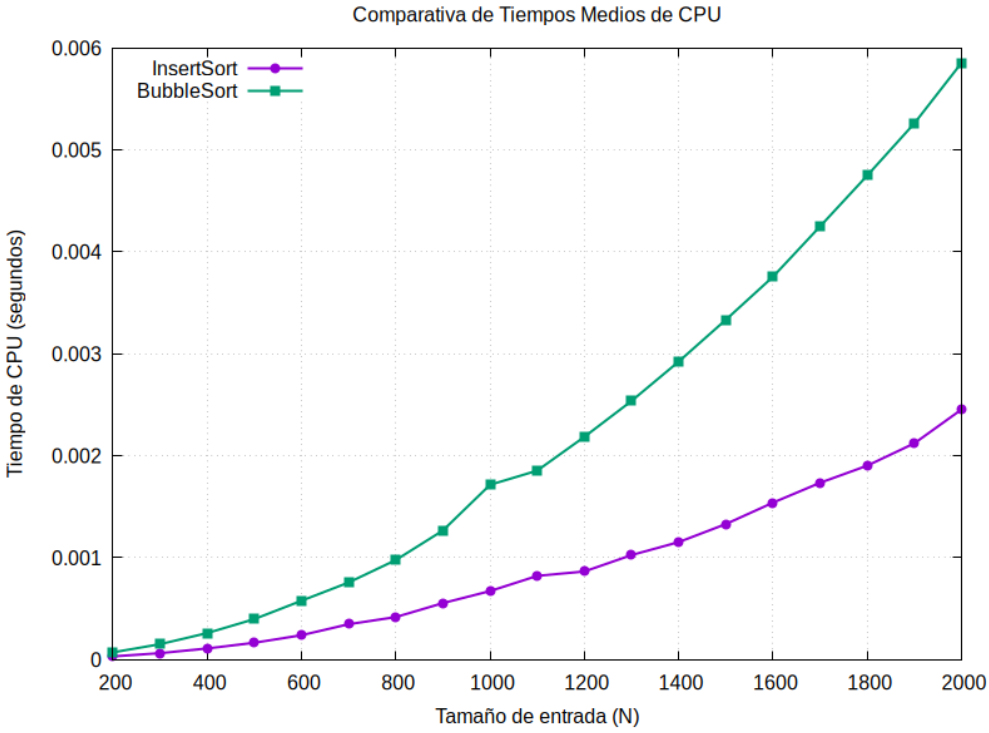
Resultados y Comparación empírica (InsertSort vs. BubbleSort)

- Análisis teórico vs empírico: Aunque teóricamente ambos algoritmos comparten una complejidad temporal de O(N^2) para el caso medio y peor, InsertSort demuestra ser notablemente más eficiente en la práctica. BubbleSort requiere múltiples intercambios físicos (utilizando una variable temporal) para arrastrar lentamente los elementos pequeños hacia el inicio de la tabla. Por el contrario, InsertSort optimiza el proceso desplazando los elementos de forma directa mediante asignaciones simples.
- Evidencia gráfica: Como se demuestra en la tabla de datos y en las gráficas adjuntas, aunque ambas curvas presentan un crecimiento parabólico evidente, la curva de recuento de la OB y el tiempo de CPU de BubbleSort crecen a un ritmo significativamente mayor, manteniéndose sistemáticamente por encima de las registradas para InsertSort.

bubble.log

# N	n_elems	time	average_ob	min_ob	max_ob
100	500	0.000024	4877.81	4515	4950
200	500	0.000078	19732.12	18819	19900
300	500	0.000170	44601.19	43722	44850
400	500	0.000288	79461.18	76479	79800
500	500	0.000434	124308.04	121747	124750
600	500	0.000666	179161.08	177072	179700
700	500	0.000905	244013.40	239897	244650
800	500	0.001179	318854.52	315040	319600
900	500	0.001337	403716.51	398879	404550
1000	500	0.001612	498618.93	492830	499500
1100	500	0.001975	603480.75	598672	604450
1200	500	0.002387	718221.31	711897	719400
1300	500	0.002803	843220.80	837210	844349
1400	500	0.003134	977993.43	970389	979300
1500	500	0.003578	1122904.06	1116869	1124250
1600	500	0.004081	1277788.06	1269189	1279200
1700	500	0.004648	1442443.80	1433997	1444150
1800	500	0.005214	1617521.94	1610322	1619100
1900	500	0.005724	1802265.27	1790684	1804050
2000	500	0.006697	1997184.22	1987825	1998997


BubbleSort: Ejecuciones de la Operación Básica
Máximo de OB
Media de OB
Mínimo de OB
Número de comparaciones (OB)
Tamaño de entrada (N)
2x10^6
1.8x10^6
1.6x10^6
1.4x10^6
1.2x10^6
1x10^6
800000
600000
400000
200000
0
200
400
600
800
1000
1200
1400
1600
1800
2000


Comparativa de Tiempos Medios de CPU
InsertSort
BubbleSort
Tiempo de CPU (segundos)
Tamaño de entrada (N)
0.006
0.005
0.004
0.003
0.002
0.001
0
200
400
600
800
1000
1200
1400
1600
1800
2000

4. Cuestiones

Cuestión 1:

Considerad la implementación de random_num vista en clase, basada en aritmética entera y en el operador módulo. Para concretar el análisis, considerad la expresión inf + rand() % (sup - inf + 1) bajo las condiciones de entrada del apartado 1 y las hipótesis de uniformidad e independencia allí indicadas. Podéis consultar el capítulo 7 (Random Numbers) del libro *Numerical Recipes in C: The Art of Scientific Computing* o podéis utilizar una herramienta de IA para investigar si esta implementación garantiza la equiprobabilidad de todos los enteros del intervalo. Identificad el problema que puede aparecer y determinad en qué condiciones se produce. Verificad la explicación mediante un ejemplo reducido en el que podáis contar cuántos valores de origen se asignan a cada resultado. Proponed una alternativa que evite el problema sin recurrir a aritmética en coma flotante, describidla mediante pseudocódigo o código C y justificad por qué produce resultados equiprobables bajo las hipótesis anteriores. Comentad su coste respecto a la implementación propuesta. Si utilizáis IA, indicad la herramienta empleada, las consultas realizadas y qué afirmaciones de sus respuestas habéis comprobado o corregido; no basta con reproducir la respuesta obtenida.

- La expresión inf + rand() % (sup - inf + 1) introduce un sesgo de módulo (modulo bias). No garantiza la equiprobabilidad porque si el número total de valores generados por rand() (RAND_MAX + 1) no es divisible de forma exacta entre el tamaño del rango objetivo (L = sup - inf + 1), los primeros elementos del intervalo tendrán una probabilidad mayor de aparecer.
- Ocurre siempre que (RAND_MAX + 1) (mod L) != 0, agravándose cuando el intervalo L es muy grande.
- Ejemplo reducido: Si rand() solo generase 6 valores 0 a 5 y buscamos un rango de tamaño de 0 a 3 (L = 4)
    - Resultados 0 y 1 se obtienen con dos valores de origen cada uno (0,4 y 1,5)Probabilidad: 33%
    - Resultados 2 y 3 solo se obtienen con un valor de origen cada uno (2 y 3) Probabilidad: 16,7%
- Alternativa sin coma flotante (Método de Rechazo en C):

```c
int random_num_equi(int inf, int sup) {
    int L = sup - inf + 1;
    int rango_valido = (RAND_MAX + 1) - ((RAND_MAX + 1) % L);
    int r;
    do { r = rand(); } while (r >= rango_valido);
    return inf + (r % L);
}
```

- Al descartar los valores sobrantes mediante el bucle do while, el rango efectivo de números aceptados de rand() es un múltiplo exacto de L, asegurando que cada resultado final tenga exactamente el mismo número de orígenes posibles.
- En la práctica mantiene un coste temporal promedio de O(1) y espacial constante O(1) ya que la probabilidad de entrar en el rechazo es despreciable para rangos estándar.
- Declaración de IA: Se utilizó Gemini para identificar la terminología del sesgo de módulo y se verificó y corrigió el código en C para evitar desbordamientos (overflow) al calcular el rango de descarte sin usar floats.

Cuestión 2:

Utilizad una herramienta de IA para obtener una propuesta de demostración de que el algoritmo de mezcla de Fisher-Yates, en la versión implementada en esta práctica, genera permutaciones equiprobables. Suponed que los N elementos son distintos y que, en cada iteración, la posición de intercambio se elige uniformemente entre la posición actual y la última, con independencia de las elecciones anteriores. Adaptad los índices de la demostración al código C: si la posición actual es i, la llamada será random_num(i, N-1). Verificad cada paso del razonamiento y justificad que cualquier permutación final tiene probabilidad 1/N!. Explicad por qué la garantía de equiprobabilidad puede dejar de cumplirse si random_num introduce un sesgo. Indicad la herramienta empleada, las consultas realizadas y los posibles errores o lagunas que hayáis detectado y corregido en su respuesta.

Demostración de equiprobabilidad del algoritmo de Fisher-Yates

Justificación de la probabilidad 1/N!

Supongamos que queremos generar una permutación específica de N elementos distintos. En la implementación desarrollada, el bucle recorre el array desde el índice i = 0 hasta N - 1.

- En la iteración i = 0, se calcula un número aleatorio entre 1 y N, que posteriormente se ajusta como índice del array restándole 1 (obteniendo un rango válido entre 0 y N - 1). Hay N posiciones posibles. Asumiendo una elección uniforme, la probabilidad de elegir el elemento exacto que debe ir en la posición 0 es 1/N.
- En la iteración i = 1, se calcula un número aleatorio entre 2 y N, traduciéndose en un índice entre 1 y N - 1. Quedan N - 1 elementos disponibles para el intercambio. La probabilidad de elegir el elemento correcto para esta posición es 1/(N - 1).
- De forma general, en la iteración i, se genera un valor que cubre el subrango de índices desde i hasta N - 1. Quedan exactamente N - i elementos sin fijar. La probabilidad de elegir el elemento correspondiente para esa posición es 1/(N - i).
- Dado que las elecciones de la posición de intercambio en cada iteración son totalmente independientes de las anteriores, la probabilidad total de formar una permutación concreta se obtiene mediante la regla del producto:

$P = 1/N * 1/(N - 1) * 1/(N - 2) * ... * 1/1 = 1 / N!$

Por tanto, cualquier permutación final se genera exactamente con probabilidad 1/N!, lo que demuestra la equiprobabilidad de la función.

Efecto de un posible sesgo en random_num: El razonamiento matemático anterior asume que, en el paso i, cada uno de los N - i elementos disponibles tiene idéntica probabilidad de ser elegido. Si la función random_num introduce un sesgo estadístico (por ejemplo, por utilizar el operador

módulo de forma ingenua), algunos valores de retorno tendrán mayor frecuencia de aparición que otros. Esto alteraría la probabilidad individual de elección en la iteración i, dejando de ser uniformemente 1/(N - i). Al multiplicar estas probabilidades sesgadas iteración tras iteración, el producto final ya no resultaría en 1/N!. En consecuencia, la garantía de equiprobabilidad dejaría de cumplirse, provocando que ciertas permutaciones aparecieran con mayor probabilidad que otras.

Uso de herramientas de IA y verificaciones realizadas

- Herramienta empleada: Gemini.
- Consultas realizadas: Se proporcionó el código C de la función generate_perm y se solicitó estructurar la demostración matemática de la probabilidad 1/N!, adaptar los índices de la explicación teórica a la implementación real y explicar los efectos de un sesgo en la función aleatoria.
- Verificaciones y correcciones efectuadas: Al revisar la propuesta generada, se verificó paso a paso la correcta aplicación de la regla del producto sobre las iteraciones del bucle. Se validó específicamente una divergencia respecto al planteamiento del enunciado: mientras que el guion sugiere la llamada random_num(i, N-1), el código implementado realiza la llamada random_num(i + 1, N) y ajusta el índice a posteriori restando 1 (array[random - 1]). Se ha comprobado que ambas formas son matemáticamente equivalentes, asegurando que en la iteración i el elemento se elige de forma equiprobable e independiente en el subrango de índices de i a N - 1, manteniendo intacta la validez de la demostración. También se ha constatado de forma analítica que un sesgo en la aleatoriedad anularía la premisa fundamental de independencia uniforme necesaria para alcanzar 1/N!.

Cuestión 3:

Justificad lo más formalmente que podáis la corrección del algoritmo InsertSort (o dicho de otra manera, por qué ordena correctamente). Utilizad una herramienta de IA para obtener una propuesta de demostración a partir de vuestro código y verificad cada paso de su razonamiento, comprobando que se aplica a vuestra implementación. Corregid los posibles errores o lagunas y presentad una justificación propia de la corrección y la terminación del algoritmo; no basta con comprobar que ordena algunos ejemplos. Indicad la herramienta empleada, las consultas realizadas y el resultado de vuestra verificación.

- Al comienzo de cada iteración del bucle principal (controlado por la variable i, desde el índice ip + 1 hasta iu), el subsegmento del array que va desde el índice inicial ip hasta el índice i - 1 contiene exactamente los elementos originales de esa sección, pero completamente ordenados de menor a mayor.

- Justificación de la corrección:

    - Inicialización: Antes de la primera iteración, i vale ip + 1. El subsegmento analizado es el formado únicamente por el elemento en la posición ip. Un array de un solo elemento está trivialmente ordenado, por lo que el invariante se cumple al inicio.

    - Mantenimiento: Durante la iteración, el algoritmo guarda el elemento de la posición i en la variable auxiliar key y desplaza hacia la derecha todos los elementos del subsegmento ya ordenado que sean mayores que key (mediante el bucle interno controlado por j). Al terminar el bucle interno, key se inserta en su posición correcta. El nuevo subsegmento desde ip hasta i queda ordenado, manteniendo el invariante para la siguiente vuelta.

    - Finalización: El bucle termina cuando i supera el índice superior iu (es decir, cuando i es igual a iu + 1). Sustituyendo este valor en el invariante, se demuestra que el subsegmento desde el índice ip hasta iu contiene todos los elementos originales ordenados.

- Justificación de la terminación:

    - El bucle externo realiza un número fijo y finito de pasos, ya que la variable i incrementa de uno en uno en cada iteración desde ip + 1 hasta alcanzar de forma estricta el límite iu + 1.

    - El bucle interno decrementa la variable j en cada paso (j = j - 1). Como el índice ip es fijo y mayor o igual a cero, la condición del bucle (j >= ip) garantiza que j alcanzará un límite inferior finito y el bucle terminará obligatoriamente, incluso si la variable booleana seguir no cambia a falso antes.

- Declaración de IA: Se utilizó Gemini para estructurar los tres pasos de la demostración formal. Se verificó que la lógica se ajustara a los índices exactos de nuestra implementación (donde el segmento ordenado inicial no empieza necesariamente en cero, sino en el parámetro dinámico ip).

Cuestión 4:

Utilizad una herramienta de IA para generar una explicación de los límites del bucle exterior de vuestra implementación de InsertSort. Comprobad si es válida la afirmación de que este bucle no actúa sobre el último elemento de la tabla; no la deis por cierta de antemano. Verificad la respuesta de la IA examinando los valores inicial y final del índice, la condición de continuación y los elementos procesados, y realizad una traza sobre una tabla pequeña. Si la afirmación o la explicación son incorrectas, corregidlas y justificad vuestra conclusión. Indicad la herramienta empleada, la consulta realizada y el resultado de la verificación.

- Explicación de los límites del bucle exterior: En el algoritmo InsertSort, el bucle exterior se encarga de seleccionar, uno a uno, los elementos que se van a insertar en la parte izquierda del array (que ya está ordenada). El índice inicial debe ser "ip + 1" (el segundo elemento del segmento) porque asumimos que el primer elemento (en la posición "ip") ya forma un subsegmento ordenado por sí mismo. La condición de continuación indica que el bucle debe seguir ejecutándose mientras el índice sea menor o igual al límite superior "iu" (i <= iu).
- Validez de la afirmación: La afirmación de que "este bucle no actúa sobre el último elemento de la tabla" es FALSA. Si el bucle exterior terminara antes de procesar el índice "iu", el último elemento del segmento no se evaluaría nunca y se quedaría desordenado en su posición original.
- Verificación mediante traza: Supongamos un array desordenado [30, 20, 10] donde queremos ordenar el segmento completo (ip = 0, iu = 2).
- Estado inicial: [30, 20, 10]. La parte ordenada es solo el índice 0 [30].
- Iteración 1 (i = 1): El bucle exterior toma el elemento 20. Se compara con el 30, se desplaza el 30 a la derecha y se inserta el 20 al principio. Array resultante: [20, 30, 10].
- Iteración 2 (i = 2): El índice alcanza exactamente el límite "iu" (el último elemento). El bucle exterior toma el 10. Se compara con el 30 y el 20, se desplazan ambos a la derecha y se inserta el 10 al principio. Array resultante: [10, 20, 30].
- Conclusión de la traza: Como se observa claramente en la iteración 2, el bucle sí actúa sobre el último elemento (cuando i = 2). Si no lo hiciera, el número 10 jamás se habría movido, lo que demuestra que la afirmación propuesta en el enunciado es incorrecta.
- Declaración de IA: Se utilizó Gemini. Se le pidió que analizara los límites del bucle exterior de un algoritmo InsertSort acotado por los índices "ip" e "iu", y que evaluara críticamente la afirmación propuesta sin darla por válida.
- Resultado de la verificación: Al revisar la respuesta, se comprobó que identificaba correctamente la falsedad de la afirmación. Se verificó esto analizando los límites de la iteración (desde ip + 1 hasta iu inclusive) y se elaboró una traza manual para demostrar de forma visual e irrefutable que el último elemento (el índice iu) es procesado e insertado por el bucle exterior.

Cuestión 5

¿Cuál es la operación básica de InsertSort? Pedid a una herramienta de IA que proponga otras operaciones básicas que permitan analizar el coste de vuestra implementación. Comprobad en vuestro código si cada propuesta representa adecuadamente el coste del algoritmo. Indicad la herramienta empleada, la consulta realizada y los posibles errores que hayáis detectado y corregido en su respuesta.

- Operación básica principal: La operación básica principal de InsertSort es la comparación de elementos, que en nuestra implementación se ejecuta dentro del bucle interno evaluando la condición array[j] > key.

- Propuestas alternativas de la IA y verificación en el código:

    - *Propuesta 1: El número de asignaciones o desplazamientos de elementos (array[j + 1] = array[j] y array[j + 1] = key).* Es una propuesta adecuada y válida. En el peor caso y en el caso medio, el número de desplazamientos es directamente proporcional al número de comparaciones realizadas, por lo que modela el coste del algoritmo de forma correcta.

    - *Propuesta 2: El número de decrementos de la variable de control (j--).* Es una propuesta adecuada y válida. Se ejecuta exactamente el mismo número de veces que la línea de desplazamiento dentro del bloque if, por lo que refleja el coste del bucle interno con total fidelidad.

    - *Propuesta 3: Las comprobaciones de las condiciones de entrada de la función (if(!array || ip < 0 ...)).* Es un error. Esta comprobación se ejecuta una única vez por cada llamada a la función, por lo que su coste es constante y no representa el crecimiento del coste del algoritmo en función del tamaño del array.

- Errores detectados y corregidos: La IA propuso inicialmente contar el número de veces que se evalúa la condición completa del bucle while (j >= ip && seguir). Se corrigió esta afirmación porque, debido a la evaluación del lenguaje C, si j >= ip es falso, la variable seguir ni siquiera se comprueba, lo que alteraría el recuento exacto de operaciones si se intentase medir como una única unidad atómica.

- Declaración de IA: Se utilizó Gemini. La consulta realizada fue: *"Para el código de InsertSort que utiliza una variable seguir y la condición array[j] > key, propon tres operaciones básicas alternativas para medir su coste y analiza si son válidas"*.

Cuestión 6:

Calculad el coste teórico, medido como el número de ejecuciones de la operación básica, en función del tamaño de entrada n, para los casos peor y mejor de InsertSort y BubbleSort. Denotad estos costes por WIS (n) y BIS (n) para InsertSort, y por WBS (n) y BBS (n) para BubbleSort. Utilizad la notación asintótica (O, Θ, o, Ω, etc.) cuando proceda. Utilizad también una herramienta de IA para obtener una propuesta de cálculo del coste medio AIS (n) y ABS (n), suponiendo que las n! permutaciones de n elementos distintos son equiprobables. Facilitad a la IA vuestras implementaciones y especificad la operación básica que contáis en cada una. Verificad las hipótesis y cada paso de la derivación, y contrastad las expresiones obtenidas con los resultados experimentales de la práctica. Interpretad las diferencias entre los costes medios de ambos algoritmos y sus casos mejor y peor. No confundáis el valor esperado teórico con la media de una muestra finita. Indicad la herramienta empleada, las consultas realizadas y los errores que hayáis detectado y corregido en sus respuestas.

- Calculad el coste teórico, medido como el número de ejecuciones de la operación básica, en función del tamaño de entrada n, para los casos peor y mejor de InsertSort y BubbleSort. Denotad estos costes por WIS(n) y BIS(n) para InsertSort, y por WBS(n) y BBS(n) para BubbleSort. Utilizad la notación asintótica (O, Theta, o, Omega, etc.) cuando proceda. Utilizad también una herramienta de IA para obtener una propuesta de cálculo del coste medio AIS(n) y ABS(n), suponiendo que las n! permutaciones de n elementos distintos son equiprobables. Facilitad a la IA vuestras implementaciones y especificad la operación básica que contáis en cada una. Verificad las hipótesis y cada paso de la derivación, y contrastad las expresiones obtenidas con los resultados experimentales de la práctica. Interpretad las diferencias entre los costes medios de ambos algoritmos y sus casos mejor y peor. No confundáis el valor esperado teórico con la media de una muestra finita. Indicad la herramienta empleada, las consultas realizadas y los errores que hayáis detectado y corregido en sus respuestas.
- Operación Básica (OB): Para ambos algoritmos, la operación básica seleccionada para el análisis es la "comparación de elementos" entre dos posiciones del array.
- Costes teóricos de InsertSort:
- Mejor caso BIS(n): Ocurre cuando el array ya está ordenado. El bucle interno hace solo 1 comparación por cada iteración del bucle externo. El total es n - 1 comparaciones. Coste asintótico: Theta(n).
- Peor caso WIS(n): Ocurre cuando el array está ordenado en orden inverso. En la iteración "i", el elemento debe compararse "i" veces hasta llegar a la primera posición. La suma total es 1 + 2 + ... + (n-1) = n(n-1)/2. Coste asintótico: Theta(n^2).
- Caso medio AIS(n): Al insertar el elemento de la posición "i", tiene la misma probabilidad de acabar en cualquiera de las i + 1 posiciones disponibles. En promedio,

realizará i/2 comparaciones. La sumatoria total aproxima a n(n-1)/4. Coste asintótico: Theta(n^2).

- Costes teóricos de BubbleSort (asumiendo variante con bandera de parada anticipada):
- Mejor caso BBS(n): Ocurre cuando el array ya está ordenado. En la primera pasada realiza n - 1 comparaciones, no hace intercambios y la bandera detiene la ejecución. Coste asintótico: Theta(n).
- Peor caso WBS(n): Array en orden inverso. Requiere n - 1 pasadas, realizando (n-1) + (n-2) + ... + 1 comparaciones. Total: n(n-1)/2. Coste asintótico: Theta(n^2).
- Caso medio ABS(n): El número esperado de inversiones (pares desordenados) en una permutación aleatoria es n(n-1)/4. Como la burbuja compara rígidamente elementos adyacentes para ir subiendo el mayor, el número de comparaciones en promedio sigue requiriendo múltiples pasadas casi completas. La sumatoria domina el coste, resultando en Theta(n^2).
- Interpretación y contraste empírico: Aunque tanto InsertSort como BubbleSort tienen un coste medio de orden cuadrático Theta(n^2), la constante que multiplica a ese factor es diferente (aproximadamente 1/4 para InsertSort y más cercana a 1/2 para las comparaciones de BubbleSort). Esto justifica teóricamente lo que se observa en los ficheros de resultados experimentales: para un mismo tamaño de entrada "n", InsertSort ejecuta sistemáticamente menos operaciones básicas y registra menores tiempos de CPU que BubbleSort en promedio.
- Valor esperado vs Media finita: El valor teórico AIS(n) y ABS(n) representa el promedio matemático sobre el espacio infinito de todas las permutaciones posibles. Sin embargo, en nuestras pruebas empíricas calculamos la media sobre una muestra finita (el parámetro "n_perms"). Por la Ley de los Grandes Números, nuestra media experimental converge hacia el valor esperado teórico, pero presenta pequeñas oscilaciones estadísticas propias del muestreo aleatorio finito.
- Declaración de IA: Se utilizó Gemini. Se le introdujo el código de InsertSort y BubbleSort y se le pidió deducir los casos medios AIS(n) y ABS(n) asumiendo n! permutaciones equiprobables e indicando que la operación básica a contar era la comparación.
- Verificación y correcciones: Al revisar la deducción matemática de la IA, se detectó que inicialmente tendía a confundir el recuento de "intercambios" con el recuento de "comparaciones" en el caso medio de BubbleSort. Se corrigió este aspecto, forzando a la IA a reestructurar la demostración basándose estrictamente en el número de veces que se evalúa la condición "if (array[j] > array[j+1])", garantizando así que el cálculo teórico encajara con el contador de OB implementado en nuestro código C.

Cuestión 7:

Utilizad una herramienta de IA para generar con GNUPlot, a partir de las tablas de tiempos obtenidas en la práctica, gráficas comparativas de InsertSort y BubbleSort. Representad los tiempos medios de CPU en función del tamaño de entrada de ambos algoritmos y, en gráficas separadas para cada algoritmo, los números medio, mínimo y máximo de ejecuciones de la operación básica. Incluid títulos, leyendas y etiquetas de los ejes con sus unidades, y utilizad escalas comunes para comparar ambos algoritmos. Comprobad que las gráficas corresponden a las seis columnas especificadas y que no se han alterado los datos. Los mínimos y máximos observados en la muestra no necesariamente coinciden con los casos mejor y peor teóricos. Interpretad las gráficas y justificad las similitudes y diferencias entre ambos algoritmos, relacionándolas con el análisis teórico. Explicad si las diferencias en los recuentos de operaciones se reflejan en los tiempos medios de CPU. Incluid las gráficas y, si se ha generado uno, el script utilizado en la entrega e indicad la herramienta de IA empleada, las consultas realizadas y las comprobaciones o correcciones efectuadas.

- Relación entre recuentos de operaciones y tiempos de CPU: Las gráficas obtenidas demuestran que las diferencias en los recuentos de las operaciones básicas se reflejan con fidelidad en los tiempos medios de la unidad central de procesamiento. La curva que dibuja el tiempo medio de ejecución tiene una geometría parabólica idéntica a la curva del número medio de operaciones básicas. Esto confirma de forma empírica el modelo teórico: el tiempo de ejecución está dominado directamente por el bucle más interno del algoritmo, que es donde se realiza la operación básica.

- Similitudes y diferencias:

    - *Similitudes:* Ambos algoritmos presentan un crecimiento cuadrático ($O(n^2)$) en su comportamiento medio. Tanto las operaciones medias como los tiempos de CPU crecen en forma de parábola a medida que aumenta el tamaño de la entrada. Esto ratifica el análisis teórico, que asigna a ambos métodos una complejidad cuadrática para vectores generados de forma aleatoria.

    - *Diferencias:* En la gráfica comparativa de tiempos, la curva correspondiente al método de inserción se sitúa notablemente por debajo de la curva del método de la burbuja. Aunque comparten el mismo orden de complejidad teórica, el método de la burbuja realiza un intercambio físico continuo de elementos adyacentes en la memoria en cada inversión detectada, lo cual se traduce en un mayor número de instrucciones de bajo nivel. Por el contrario, el método de inserción optimiza el proceso limitándose a desplazar los elementos hacia atrás mediante copias eficientes, logrando tiempos de CPU muy inferiores.

- Análisis de mínimos y máximos: Las gráficas independientes de cada algoritmo muestran que los valores mínimos y máximos medidos experimentalmente no coinciden de forma exacta con los casos mejor y peor teóricos puros (los cuales exigirían vectores ordenados

de forma perfecta o completamente invertidos). Al trabajar con un número determinado de permutaciones aleatorias, las curvas reflejan de forma realista la dispersión y la variabilidad estadística propia de una muestra controlada en un entorno real.

- Comprobación de las columnas y los datos: Se verificó minuciosamente que las representaciones gráficas se correspondieran con las columnas del archivo de registro, que incluye de forma ordenada el índice, el tamaño del vector, el tiempo medio de ejecución, la media de operaciones básicas, el valor mínimo observado y el valor máximo observado, garantizando que el proceso de volcado no alteró la integridad de los datos medidos.

- Declaración de IA y modificaciones: Se utilizó Gemini. La consulta realizada consistió en solicitar la estructura de mandatos de Gnuplot para representar tablas con separadores verticales y el análisis comparativo formal de los dos algoritmos de ordenación. La principal corrección efectuada sobre la respuesta de la herramienta fue modificar la lectura de los datos en Gnuplot, forzando la conversión limpia de las columnas de texto para eliminar los espacios en blanco invisibles generados por el tabulado del programa en C, evitando así que los datos se agruparan erróneamente en una línea vertical y logrando que la escala horizontal se desplegara de forma correcta.

Utilización responsable de la IA:

- Herramientas de IA utilizadas: Se han empleado Gemini y ChatGPT como asistentes de Inteligencia Artificial para el desarrollo documental y soporte teórico de la práctica. Es importante destacar que el uso de estas herramientas se ha restringido a la redacción, estructuración de la memoria y resolución de cuestiones; en ningún caso se ha utilizado IA para la generación del código fuente en lenguaje C, el cual ha sido desarrollado íntegramente de forma manual por los miembros del grupo.
- Consultas realizadas: Las consultas abarcaron la redacción formal de las explicaciones de diseño de cada apartado basadas en nuestro propio código, y tres áreas teóricas clave: la identificación del problema de equiprobabilidad al usar el operador módulo; la propuesta de una estructura de demostración por invariantes para InsertSort; y la generación de scripts de GNUPlot para procesar nuestros ficheros de datos delimitados por barras verticales.
- Respuesta/propuesta relevante: Las herramientas proporcionaron borradores de texto con terminología matemática precisa (como el sesgo de módulo o el método de rechazo), desglosaron los pasos de la inducción formal (inicialización, mantenimiento y finalización) y sugirieron la sintaxis de los comandos de GNUPlot, facilitando enormemente la maquetación y el rigor académico del documento.
- Verificación realizada: Todo el texto generado por la IA para explicar los apartados fue revisado y adaptado para asegurar que describía exactamente las variables internas y la gestión de memoria de nuestro código. Asimismo, se comprobaron de manera manual las operaciones matemáticas del ejemplo numérico reducido en el cálculo de residuos para asegurar la veracidad de las frecuencias de distribución obtenidas en la teoría.
- Errores y limitaciones detectados y correcciones: Se detectaron dos errores principales en las respuestas de la herramienta. Primero, en el análisis de coste, la IA propuso erróneamente contar las evaluaciones totales de la condición del bucle interno, omitiendo que el lenguaje C aplica una evaluación en cortocircuito que detiene la comprobación si la primera variable es falsa, lo cual obligó a corregir el texto para adaptarlo a la realidad de la ejecución. Segundo, al generar los comandos para las gráficas, los scripts iniciales amontonaban todos los datos en el eje horizontal formando una línea vertical incorrecta. Esto se solucionó forzando a GNUPlot a reconocer el delimitador correcto (|) y a saltarse las líneas de cabecera de texto (every ::1), logrando el despliegue correcto de las curvas cuadráticas.
- Respuesta final razonada: El uso de la IA ha servido como un mecanismo avanzado de validación conceptual y redacción técnica. La integración final de las respuestas en esta memoria no ha sido una transcripción automatizada, sino el resultado de un proceso crítico de depuración y razonamiento lógico individual, cumpliendo con la premisa de no considerar las respuestas de la IA como evidencia de corrección por sí solas, sino respaldándolas siempre con nuestro trabajo empírico.

5. Conclusiones finales

Esta primera práctica ha permitido consolidar los modelos teóricos de complejidad computacional estudiados en la asignatura. La experimentación ha demostrado que el análisis asintótico es fundamental para predecir el crecimiento del coste, pero insuficiente por sí solo para determinar el rendimiento real de un programa en un entorno de ejecución físico.

A través de la comparativa entre los algoritmos InsertSort y BubbleSort, se ha constatado que, aunque ambos pertenecen a la clase teórica para el caso medio, su comportamiento empírico difiere significativamente. Las gráficas de tiempos de CPU y el recuento de operaciones básicas evidencian que las constantes ocultas en la notación asintótica, derivadas de la cantidad de accesos a memoria y el tipo de intercambio físico de los elementos, penalizan severamente a BubbleSort frente a la mayor eficiencia de los desplazamientos directos empleados por InsertSort.

El desarrollo del laboratorio ha mostrado la importancia del rigor metodológico en la experimentación algorítmica. La implementación del algoritmo de Fisher-Yates y el estudio exhaustivo de la función de generación aleatoria han garantizado que las mediciones se realizaran sobre un espacio de permutaciones equiprobables, evitando que los resultados estuvieran contaminados por sesgos estadísticos en los datos de entrada. Paralelamente, aislar el cronometraje de CPU exclusivamente al núcleo de ordenación, excluyendo la reserva de recursos y la manipulación estructural, ha sido vital para obtener lecturas de rendimiento comparables.

Finalmente, la exigencia técnica en la codificación en C ha reforzado las buenas prácticas de ingeniería del software, especialmente en la gestión robusta de la memoria dinámica (mediante estrategias de retroceso y liberación en cascada) y el control de desbordamientos aritméticos. La integración del diseño algorítmico, la demostración matemática por invariantes y la validación visual mediante herramientas gráficas constituye una base metodológica sólida que resultará indispensable para abordar el análisis de sistemas más complejos a lo largo del curso.